

Real-Time Juggling Counter

Introduction to Image Processing

Final Project Report

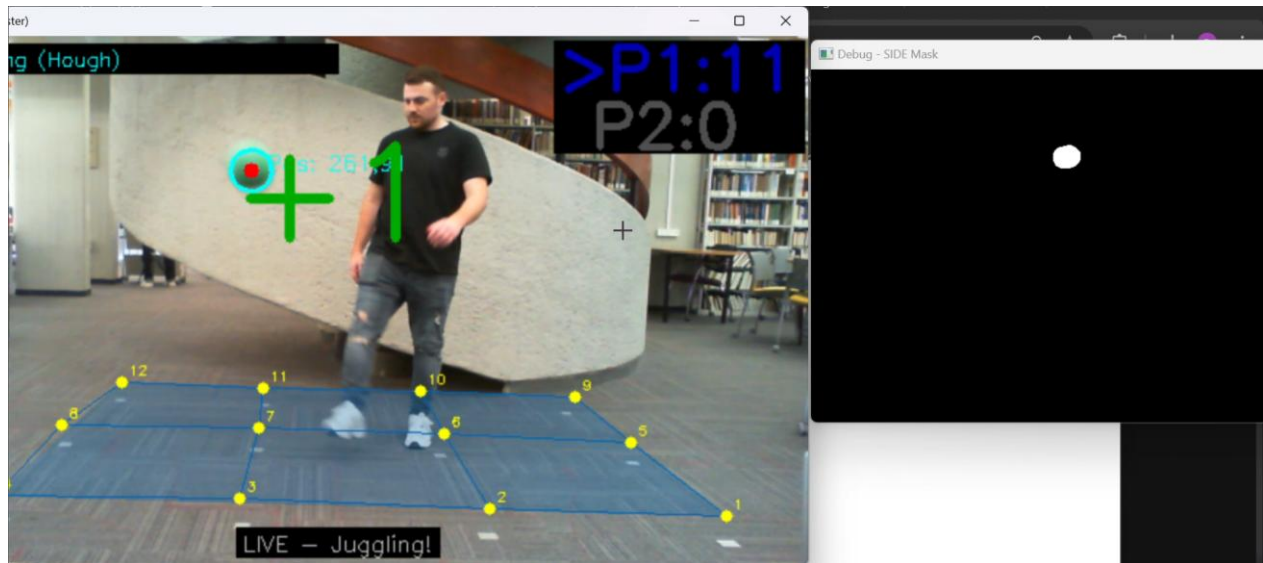
Professor Tami Riklin

Project Team

Bar Megidish		Eylon Oren		Itamar Bahat		Yosef Shalev
207798588		318157419		205805856		208472316

March 2026

[\(Link to the video\)](#)



1. Introduction

This project implements a real-time juggling counter that automatically detects, tracks, and counts soccer ball juggles using computer vision. The system provides immediate visual feedback, detects drops, and supports two-player gameplay.

1.1 System Assumptions

To ensure the integrity of the tracking and counting logic, the following assumptions were made:

- **Static Camera Positioning:** Both the Side and Top cameras are assumed to be stationary throughout the session to maintain consistent spatial calibration.
- **Object Sphericity:** The movement analysis and Radius Gating logic rely on the ball maintaining a near-spherical silhouette during flight.
- **Color Contrast:** The system assumes a distinct color difference between the ball and the background environment for effective HSV thresholding (another distinction effect will be introduced).
- **Constant Lighting:** Minimal variation in ambient light is assumed to prevent drastic shifts in the pre-calibrated HSV bounds.
- **Kinematic Mimicry:** We assume that human juggling follows a predictable physical pattern of velocity reversal. This creative premise allows the system to accurately count hits by detecting kinematic 'V-Flips' rather than relying on computationally expensive geometric collision modeling.

2. Pipeline Overview

Each frame from both cameras passes through the same processing stages before an event (HIT or DROP) is produced. The diagram below illustrates the full pipeline.



3. Ball Detection

The detection pipeline transforms raw camera frames into a validated ball position (x, y, r).

3.1 Gaussian Blur

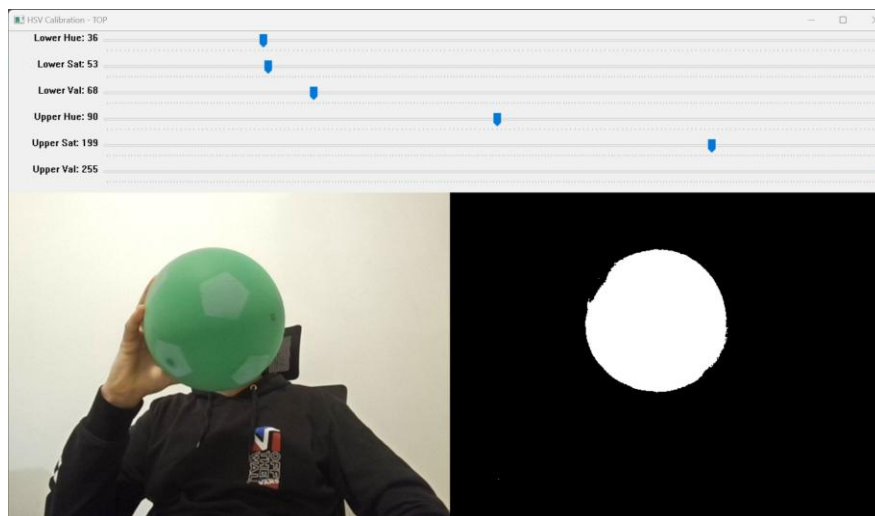
- **What it does:** Applies a Gaussian kernel (11x11) to smooth the image before any analysis.
- **Why we chose it:** Reduces high-frequency sensor noise.
- We selected an 11x11 kernel through empirical testing as the "sweet spot" for our camera resolution. It is large enough to suppress high-frequency sensor noise in indoor lighting, yet small enough to prevent the ball's edges from bleeding into the background, which would degrade the Hough Circle's precision.

3.2 Background Subtraction (MOG2)

- **What it does:** An adaptive Gaussian Mixture Model continuously learns the static scene. Pixels that deviate from the learned background are classified as foreground (moving).
- **Why we chose it:** MOG2 adapts to gradual lighting changes and small scene variations. It reliably separates moving objects from the static environment without manual recalibration.
- Operates on the assumption that background pixels change slower than foreground objects. The learning rate is the critical parameter; We optimized it to a stage where the ball (while moving) remains classified as the dominant foreground object in the scene and "ghosting" artifacts are quickly absorbed into the background model.

3.3 HSV Color Thresholding

- **What it does:** The frame is converted from BGR to HSV and thresholded with calibrated bounds to produce a binary mask of pixels matching the ball's color.
- **Why we chose it:** HSV color space separates color (Hue) from brightness (Value). Isolating the Hue channel, making the detection robust to shadows and lighting variation.



3.4 Bitwise AND Fusion

- **What it does:** The motion mask and color mask are combined with a bitwise AND operation. Only pixels that are both in the foreground AND match the ball's color survive.
- **Why we chose it:** Color alone detects static objects with the same color, motion alone detects moving body parts. Combining them eliminates both failure modes.

3.5 Morphological Operations (Opening & Closing)

- **What it does:** Opening (erosion then dilation) removes small noise. Closing (dilation then erosion) fills gaps inside the ball's silhouette. The result is a clean binary mask.
- **Why we chose it:** The fused mask contains residual noise (small, isolated pixels) and small holes within the ball region.

3.6 Hough Circle Transform

- **What it does:** Searches the binary mask for circular shapes within a calibrated radius range. Returns the center and radius of the strongest detected circle.
- **Why we chose it:** Only truly round shapes are accepted. This prevents body parts, shoes, or irregular blobs from being mistaken for the ball.
- To ensure distinct separation between the ball and other objects, we optimized the param2 parameter (Accumulator threshold) via a dedicated configuration file. This parameter dictates the required circularity level. by fine-tuning it, we achieved an optimal balance between detection sensitivity and robustness against false positives from body parts or static clutter.



3.7 Contour Analysis

- **What it does:** If Hough fails on the secondary camera, the system finds all contours in the mask, selects the largest by area, and fits a minimum enclosing circle.
- **Why we chose it:** When the ball appears small or partially occluded (e.g., near the floor from the side angle), it may not form a perfect circle.

3.8 Kalman Filter

- **What it does:** A linear Kalman Filter with a 4D state vector $[x, y, dx, dy]$ is updated with each detection. When the ball is lost, it predicts the next position based on last known velocity. Each camera runs its own independent Kalman filter.
- **Why we chose it:** During brief occlusions (e.g., ball passes behind a leg), detection is lost for a few frames. The Kalman filter bridges these gaps by predicting the ball's position from its trajectory.
- **Implementation & Impact (occlusion handling):** Based on a constant-velocity kinematic model assumption between frames, we tuned the Process Noise Covariance to balance smoothness with responsiveness. This configuration bridges up to 10 consecutive frames of occlusion without introducing tracking "lag" during kicks, ensuring the state machine maintains its "falling/rising" status even when the ball is temporarily hidden and preventing an accidental drop trigger.

4. Movement Analysis & Impact Detection

Once a ball position is extracted, the system analyzes vertical motion from the main camera to detect juggle kick events.

4.1 Radius Gating

- **What it does:** Compares the detected radius to a calibrated baseline. Detections deviating beyond $\pm 50\%$ are rejected as false positives.
- **Why we chose it:** Radius gating provides a fast sanity check that eliminates obvious false detections.
- it filters out non-ball objects (e.g., shoes or hands) that might temporarily pass color and motion filters, ensuring the system only tracks objects of the expected physical scale.

4.2 Finite-Difference Velocity Estimation

- **What it does:** Computes vertical velocity as $\Delta Y = Y_{\text{current}} - Y_{\text{previous}}$ between consecutive frames. Positive values indicate falling, negative values indicate rising.
- **Why we chose it:** A simple and effective approach for real-time velocity.

4.3 Inertia State Machine

- **What it does:** Maintains two counters: consecutive falling frames and consecutive rising frames. A direction change resets the opposite counter. After 2+ falling frames, the state locks to "IS_FALLING".
- **Why we chose it:** A single frame of upward motion could be noise or jitter. Requiring multiple consecutive frames of consistent motion eliminates transient false triggers and provides robust state tracking.
- The system requires 2+ frames of consistent motion to lock a state, effectively acting as a temporal low-pass filter that ignores transient noise and jitter. This specific threshold (2 frames) was selected as an optimal balance: it is robust enough to eliminate false triggers while remaining sensitive enough to capture rapid, high-speed juggling and quick direction changes in real-time.

4.4 Velocity Reversal Detection ("V-Flip")

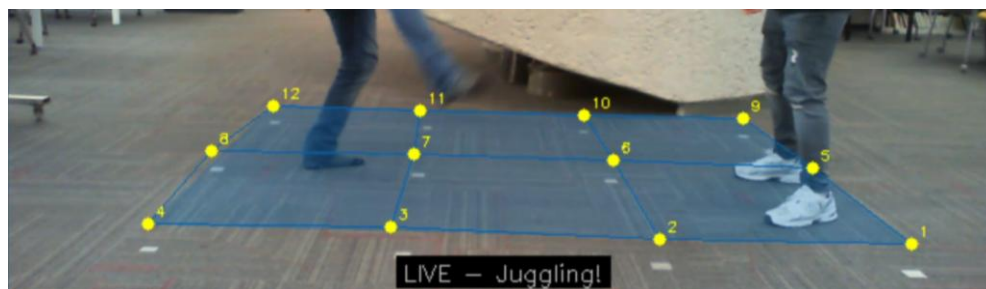
- **What it does:** A HIT is registered when the state was IS_FALLING and then 2+ consecutive rising frames are observed. For the first kick (flick-up from the floor), stricter thresholds apply: 3+ rising frames and a minimum displacement of 15 pixels (made to take into account different starting conditions).
- **Why we chose it:** A juggle kick creates a characteristic V-shaped trajectory: the ball falls, contacts the foot, then rises. Detecting this reversal pattern is a direct physical model of a kick, making it highly specific and resistant to false positives.
- **Note:** instead of attempting to model the complex contact geometry between the foot and the ball—a computationally intensive task for real-time systems; we adopted a creative approach based on rigid-body kinematics. The V-Flip serves as a direct physical model of the impact's outcome. This allowed us to transform a sophisticated geometric problem into an efficient vector analysis of velocity reversal, significantly enhancing system stability and speed.

5. Floor Detection & Drop Logic

The system is designed with a modular, multi-layered architecture to support safety layers for high-precision counting. Each independent layer is used to determine if the ball has touched the floor. This infrastructure allows for the integration of kinematic analysis, energy-decay signatures (bounce detection), and geometric constraints within a shared verification pipeline, and ensures robust cross-validation. This multi-layered approach maintains tracking integrity even under optical noise, effectively preventing false hits or erroneous drop detections.

5.1 Dual-Camera Homography

- **What it does:** A 12-point calibration maps pixel coordinates from each camera to real-world floor positions. A homography matrix (H) is computed per camera via RANSAC. Each frame, both cameras perspective-transform the ball's center onto the shared ground plane. If the Euclidean distance between projections is below an epsilon threshold, the ball is confirmed on the floor.
- **Why we chose it:** A single camera cannot reliably distinguish a ball near the floor from one slightly above it, due to perspective ambiguity. By using two cameras and projecting onto a shared world plane, the system exploits the geometric constraint that only a ball truly on the floor will produce matching projections from both viewpoints. This is the primary reason the secondary camera is angled toward the floor.



5.2 Boundary Check

- **What it does:** If the ball's X coordinate in the main camera falls outside the defined left/right boundaries, an immediate DROP is triggered.

5.4 Velocity Freeze Detection

- **What it does:** If the ball's velocity stays below a jitter threshold for ~30 consecutive frames (~1 second), the ball is classified as stationary and a drop event is initiated.

5.5 Tracking Loss Timeout

- **What it does:** If both cameras report no detection for ~30 frames (~1 second), the game ends. If the ball was last seen high up with upward velocity, the timeout extends to ~90 frames (~3 seconds).
- **Why we chose it:** If both cameras lose the ball entirely, the game cannot continue. A dynamic timeout with high-kick immunity avoids false drops during high kicks where the ball temporarily leaves the frame.
- This dynamic threshold was initially designed to support Top-Camera integration for header detection, an extension for the gameplay, to allow the player to juggle with his head. It operates in coordination with the side cameras but utilizes independent logic based on 'Radius-Flip' ((detecting size changes relative to the lens). This allows parallel processing where each camera monitors a specific height zone, mitigating occlusions and enabling seamless transition between foot juggles and headers.

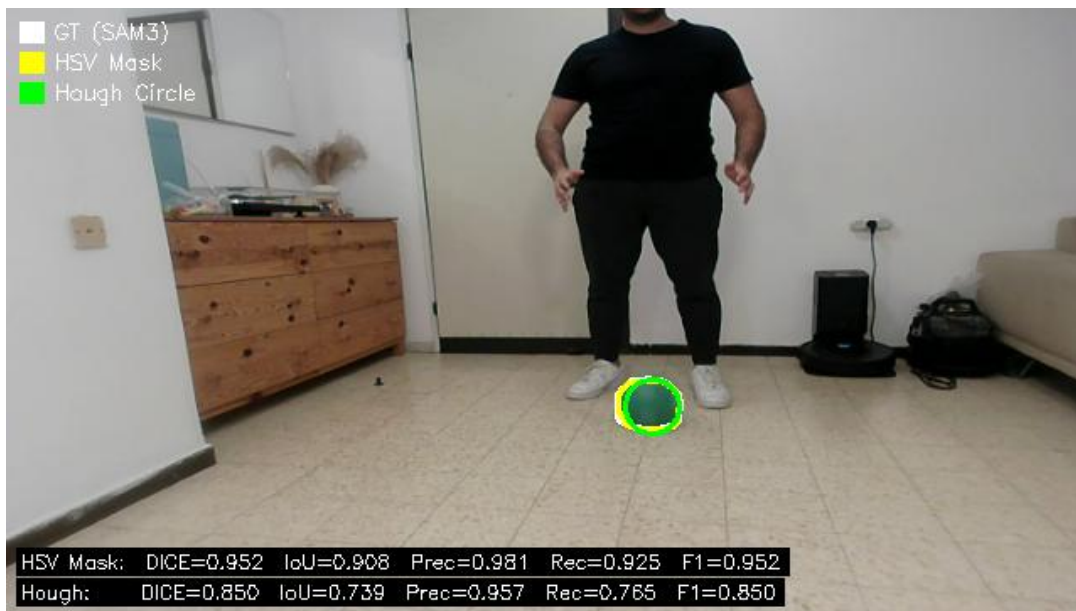
5.6 Retroactive Kinematic Analysis

- **What it does:** Acts as a retroactive "safety wrapper" for the counting logic and Examines the time intervals between recent hits. If the ball remains on the floor, this layer identifies rapid, physically impossible hit patterns and subtracts them from the score to ensure integrity.
- **Why we chose it:** This was our initial non-geometric design for floor detection. While we later implemented a primary Homography-based floor for a more responsive and "live" user experience and robust performance, we retained this kinematic layer as a critical backup to handle edge cases like floor-vibration "ghost hits".
- **Hidden Assumptions & Parameter Impact:** Assumes that human reaction time for a juggle exceeds 0.35 seconds. By monitoring the energy-decay signature (decreasing intervals between hit timestamps), the system distinguishes between an intentional kick and a passive bounce.

6. Quantitative Evaluation

To quantitatively assess the performance of our classical image processing pipeline, we compared its output against masks generated by SAM3 (Segment Anything Model 3), a state-of-the-art semantic segmentation model, SAM3 accepts a text prompt (in our case, "ball") and produces precise instance segmentation masks, which we use as our ground truth reference.

Our motivation was to measure how well a classical computer vision pipeline, built entirely on traditional techniques, performs relative to modern deep learning models.



6.1 Evaluation Setup

We captured 12 frames from our juggling video covering diverse conditions, Each frame was processed through our pipeline (Gaussian blur, HSV color thresholding, Morphological operations, Hough Circle Transform).

Note: Since we evaluate based on individual frames rather than a continuous video stream, temporal components such as the MOG2 background subtractor are not active.

We compare two outputs of our pipeline against the SAM3 ground truth:

- Raw HSV Mask - the binary segmentation mask after color thresholding and morphology.
- Hough Circle Mask - a filled circle derived from the Hough Circle Transform detection.

6.2 Metrics

For our use case, detecting a small ball in a large frame, DICE and IoU are the most informative metrics. DICE and IoU directly measure the overlap between predicted and ground-truth ball regions, making them the most honest indicators of segmentation quality. Precision and Recall are also valuable, precision tells us what fraction of our detected pixels are truly ball pixels, while Recall tells us what fraction of the actual ball we successfully detected.



6.3 Results

Evaluated on 12 frames:

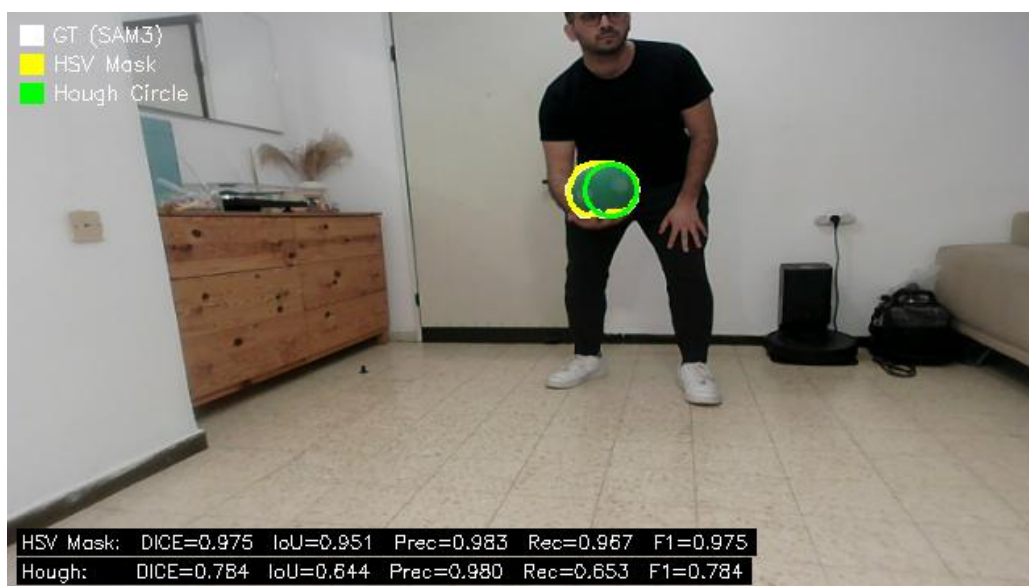
Metric	Raw HSV Mask (mean $\pm$ std)	Hough Circle (mean $\pm$ std)
DICE	0.953 $\pm$ 0.011	0.797 $\pm$ 0.085
IoU	0.912 $\pm$ 0.021	0.665 $\pm$ 0.112
Precision	0.955 $\pm$ 0.030	0.756 $\pm$ 0.176
Recall	0.953 $\pm$ 0.025	0.878 $\pm$ 0.093
F1	0.953 $\pm$ 0.011	0.797 $\pm$ 0.085
Accuracy	0.999 $\pm$ 0.000	0.996 $\pm$ 0.005

6.4 Analysis

The raw HSV mask achieves a mean DICE of 0.953 and IoU of 0.912, indicating that our classical color-based segmentation captures over 91% of the ball region with high overlap against the SAM3 reference.

The Hough Circle detection shows lower overlap scores (DICE 0.797, IoU 0.665), which is expected - the Hough Transform fits an idealized circle to the detected region, while the actual ball shape in the image may be slightly elliptical.

We are satisfied with these results. A DICE score above 0.95 for the raw mask demonstrates that our classical pipeline segments the ball with accuracy approaching that of a deep learning model, while operating at real-time speeds without requiring GPU acceleration. The Hough-based localization, while geometrically approximate, achieves 100% detection rate and provides the center-and-radius representation needed for our tracking and game logic. This trade-off ensures the system remains accessible on standard hardware while maintaining the accuracy required for professional gameplay.



7. Conclusion

Working on this project was a genuinely rewarding experience. What began as a seemingly straightforward task-counting juggles, quickly revealed the depth and difficulty of real-time image processing.

Challenges emerged at every stage - detecting a round object against cluttered backgrounds, resolving perspective ambiguity- whether the ball touched the ground or just hovered close to it, and handling the inevitable occlusion moments where the ball simply disappears from cameras site while play continues. Addressing those demanded integration of spatial geometry, kinematic modeling, and robust code architecture to handle the unpredictable edge cases of live gameplay. Reframing these challenges into measurable physical constraints-rather than mere visual patterns, is what ultimately enabled a stable and reliable system.

To tackle those challenges, we broke the problem into focused sub tasks - ball detection, movement tracking, kick recognition, and drop detection, and addressed each one with a redundant, multi-layered pipeline.

MOG2 isolates motion, HSV isolates color, and their fusion filters what neither could alone. A single camera struggles with depth, so a second camera and homography resolve it.

At every level, one method backs up another.

The key lesson we learned was the need to stop approaching the problem the way we instinctively do as humans.

We don't "see" a kick happening, we detect a velocity reversal, we don't "recognize" the floor we measure whether two projected points agree geometrically.

Reframing each challenge into something a 2D camera and basic math can reliably measure, rather than imitating human perception, is what ultimately made the system work.

Bibliography

- Zivkovic, Z. (2004). "Improved adaptive Gaussian mixture model for background subtraction." ICPR. (Background for MOG2).
- Yuen, H. K., et al. (1990). "Comparative study of Hough transform methods for circle finding." Image and Vision Computing.
- Welch, G., & Bishop, G. (1995). "An Introduction to the Kalman Filter." UNC-Chapel Hill.
- Kirillov, A., et al. (2023). "Segment Anything." arXiv preprint. (Reference for SAM3).